PETROLEUM DEALER LOGISTICS AI PLATFORM

Comprehensive Viva Examination & System Architecture Preparation Guide

Project: Petroleum Dealer Decision Intelligence & Logistics AI	**Stack:** React 18, Vite, FastAPI, PostgreSQL, SQLite, Docker
Core Focus: Multi-Agent Decision Engine & Supply Chain Optimization	**Dataset:** 350-Order Empirical Dataset (K-Means & Factor Analysis)

1. Executive Overview & Problem Statement

Petroleum retail outlets operate in high-volatility, low-margin, and heavily regulated supply environments. A typical petroleum dealer manages underground storage tanks (Petrol/MS and Diesel/HSD) that must never run dry (causing massive customer dissatisfaction and regulatory non-compliance) nor overfill (causing catastrophic environmental hazards). Dealers struggle with complex procurement decisions: when to order, how much to order, which refinery slot to book, which tanker to assign, and which driver has valid statutory hazardous materials endorsements. This project develops an end-to-end **Multi-Agent Decision Intelligence Platform** that combines **deterministic mathematical logistics guardrails** with **predictive machine learning models** to automate, audit, and explain supply chain operations.

2. Frontend Architecture & Technical Implementation

The frontend is an enterprise Single-Page Application (SPA) designed to give dealership managers instant visibility and interactive AI decision support.

Core Frontend Stack:

- **React 18.2:** Component-based declarative UI library utilizing modern functional components and hooks (useState, useEffect, useContext, useMemo).
- **Vite 5.4:** Next-generation frontend build tool providing Hot Module Replacement (HMR) and an ultra-fast esbuild bundler.
- **TailwindCSS 3.4:** Utility-first styling engine configured with an industrial dark/slate design system, semantic status tokens (good, warn, bad), and custom typography.
- **React Router DOM 6:** Declarative client-side routing with route-level authentication guards (ProtectedRoute) and role inspection.
- **Axios HTTP Client:** Centralized HTTP service with request/response interceptors automatically injecting JWT Bearer tokens and parsing backend error schemas.

Key Frontend Pages & Modules:

Page / Component	Architectural Purpose & Technical Details
Dashboard.jsx	Executive operational cockpit. Fetches from /analytics/{id} and /inventory/{id}/risk. Shows Petrol/Diesel stock, pending orders, fleet utilization %, critical expired alerts, and an auto-generated executive summary.
Inventory.jsx	Live tank monitor. Calculates fill percentage (current_level_l / tank_capacity_l) * 100, renders dynamic color-coded progress bars (Red <25%, Amber <50%, Green >50%), days of stock remaining, and reorder points.
LogisticsDecisionCard.jsx	The core AI recommendation view. Explains multi-fuel priority rankings, safety stock deficit, recommended vs min/max order volumes, tanker/driver suitability audit, and consequences analysis ('What happens if I wait 2 days?').
AIAssistant.jsx	Conversational AI interface. Dispatches queries to /ai/query, displays AI answers alongside structured tool execution traces, verified data sources, and 1-click 'Generate Order' staging action.
Orders.jsx & Dispatch.jsx	Full supply cycle lifecycle management: Fuel Orders (PENDING → APPROVED → FULFILLED) and Dispatch Plans (PLANNED → ACTIVE → DELIVERED) with live tanker/driver status changes.
Compliance & Fleet	Audits statutory licenses: Explosive Rules, PESO certification, Pollution Under Control (PUC), and Hazardous Goods Driver Endorsements.

3. Backend Architecture & High-Performance Design

The backend is developed in **Python 3.12** using **FastAPI**, adhering to Clean Architecture and Domain-Driven Design (DDD) principles.

Core Backend Stack:

- **FastAPI:** Modern, asynchronous web framework built on Starlette (ASGI) and Pydantic. Delivers native JSON validation, sub-millisecond route handling, and automatic OpenAPI/Swagger documentation at /docs.
- **SQLAlchemy 2.0 ORM:** Object-Relational Mapping layer providing type-safe database queries, connection pooling, transactional integrity, and database portability between SQLite (local development) and PostgreSQL (Docker production).
- **Pydantic v2:** Data validation and serialization engine. Enforces strict schema typing for incoming payloads and outbound responses (e.g., AnalyticsOut, InventoryRiskOut, OrderCreate).
- **Uvicorn:** Lightning-fast ASGI server implementation powered by uvloop and httptools.
- **PyJWT & Passlib (Bcrypt):** Implements stateless JWT bearer token authentication, secure salt generation, and SHA-256 password hashing.

Database Schema & Relational Architecture:

The database contains 11 normalized relational tables with enforced Foreign Key constraints and unique indexes: **dealers** (outlet identity & coordinates), **users** (role-based credentials), **daily_sales** (historical demand log), **fuel_inventory** (underground tanks with unique [dealer_id, fuel_type] constraint), **fuel_orders** (purchase requisitions), **tankers** (fleet assets, capacities, status), **drivers** (personnel & hazmat license expiry), **refineries** (terminals & default distances), **loading_slots** (depot gantry booking windows), **dispatches** (active delivery execution plans), and **compliance_records**.

4. The Multi-Agent Decision Intelligence Engine

Unlike naive chatbots that let Large Language Models (LLMs) hallucinate supply chain numbers, this platform utilizes a **Multi-Agent System (MAS)** with **Deterministic Mathematical Guardrails**. LLMs are restricted strictly to language parsing and explanation; all quantities, risks, and routes are calculated by specialized deterministic agents.

Specialized Agent	Mathematical / Algorithmic Responsibility
InventoryAgent	Calculates Days of Stock = Current Stock / Predicted Daily Demand. Categorizes status into CRITICAL (≤1.5 days), WARNING (≤3.0 days), and NORMAL. Calculates stockout probability using Gaussian Error Integrals.
DemandForecastAgent	Machine Learning demand forecasting. Employs Random Forest / Ridge Regression trained on historical sales, seasonality (day-of-week, month), and holiday features to forecast next 1-7 day consumption.
FleetIntelligenceAgent	Tanker allocation algorithm. Filters available tankers, matches tank volume against order volume, calculates ullage headroom, and scores utilization percentage: (order_volume / tanker_capacity) * 100.
ComplianceAgent	Statutory audit engine. Blocks dispatch if driver hazardous endorsement is expired or tanker hydrostatic pressure test / PESO certificate is within 7 days of expiry.
WeatherTrafficAgent	Calculates route transit delay multipliers. Synthesizes precipitation, congestion factors, and peak gantry queues. Discloses whether source data is LIVE API or SYNTHETIC fallback.
LogisticsDecisionEngine	Top-level orchestrator. Resolves multi-fuel priority, clamps order quantity to [min_safe_qty, tank_capacity - current_inventory], conducts consequence analysis ('What if I wait 2 days?'), and generates ranked decisions.
AIAssistant	Hybrid RAG and intent router. Classifies queries, delegates to agents, verifies tool execution traces, and composes natural language explanations grounded purely in verified database metrics.

5. Comprehensive Glossary: Difficult & Technical Terms

This section prepares you for tough viva questions on domain terminology, mathematical modeling, and distributed systems engineering.

Term	Category	Technical Definition & Project Significance
Ullage (Tank Headroom)	Petroleum Domain	The empty, unfilled space in a storage tank: Ullage = Tank Capacity - Current Inventory. In petroleum logistics, ordering more than the available ullage causes tank overflow, which is a catastrophic fire hazard and environmental violation. The system strictly clamps orders to ≤ Ullage.
Safety Stock Buffer	Supply Chain	The minimum reserve inventory kept to cushion against demand spikes or transit delays: SS = Z × σ_L. If current stock drops below safety stock, the platform triggers a CRITICAL priority alert.
Reorder Point (ROP)	Supply Chain	The inventory threshold that triggers a replenishment order: ROP = (Average Daily Demand × Lead Time) + Safety Stock. When stock ≤ ROP, an order recommendation is staged.
Normal CDF (Φ(Z))	Mathematics / Probability	Cumulative Distribution Function of the standard normal distribution. In logistics_engine.py, it calculates the exact mathematical probability of running out of stock before delivery arrives: P(Stockout) = Φ((Expected Demand - Stock) / StdDev).
Cluster Analysis (K-Means)	Machine Learning	Unsupervised learning algorithm partitioning the 350-order empirical dataset into *k* distinct operational segments based on distance, volume, delay, and stockout risk. Evaluated using Euclidean distance and inertia minimization.
Silhouette Score	Statistical Validation	Metric measuring cluster cohesion vs separation, ranging from -1 to +1: s = (b - a) / max(a, b). Confirmed optimal separation across operational archetypes (e.g. Urban High-Volume vs Remote Critical-Risk).
Factor Analysis (FA)	Multivariate Statistics	Dimensionality reduction technique that uncovers unobserved latent factors (e.g., Transit Impedance Factor, Fleet Stress Factor) underlying observed correlations, rotated via Varimax rotation.
Deterministic Guardrails	AI Architecture	Architectural barrier ensuring that critical business decisions (liters ordered, monetary costs, tank limits) are computed by deterministic mathematical functions, preventing generative AI from 'hallucinating' unsafe values.
JWT (JSON Web Token)	Web Security	Stateless authentication standard consisting of three Base64URL parts: Header.Payload.Signature. Signed with HMAC-SHA256, allowing the backend to verify dealer credentials without constant database session lookups.
ASGI vs WSGI	Backend Architecture	WSGI (Web Server Gateway Interface) is synchronous and blocks worker threads. ASGI (Asynchronous Server Gateway Interface), used by FastAPI/Uvicorn, supports non-blocking async I/O, WebSockets, and high-concurrency request handling.
Vite Reverse Proxy	Frontend Networking	Development server configuration in vite.config.js that rewrites /api/* requests to http://localhost:8000, eliminating Cross-Origin Resource Sharing (CORS) preflight failures and cookie restrictions.
Containerization & Docker Compose	DevOps / Infrastructure	Packaging software and dependencies into isolated Linux containers. The project's docker-compose.yml coordinates three isolated micro-services (Postgres DB, FastAPI Backend, React Frontend) on an internal bridge network.

6. Top 15 Viva Questions & Faculty Defense Answers

Here are the exact technical questions faculty examiners frequently ask, paired with concise, authoritative model answers.

Q1. Why did you choose FastAPI over Flask or Django for the backend?

Answer: FastAPI offers three distinct advantages for this system: (1) **High Concurrency & Speed:** Being built on Starlette and ASGI, it handles asynchronous operations natively at speeds comparable to NodeJS and Go; (2) **Strict Data Validation:** Built-in Pydantic integration ensures all logistics quantities and parameters are strictly validated at runtime with zero boilerplate; (3) **Self-Documenting API:** It automatically generates interactive OpenAPI/Swagger documentation, accelerating frontend-backend integration and automated API testing.

Q2. Why not let an LLM (like GPT-4) calculate the recommended order volume directly?

Answer: LLMs are probabilistic language models prone to hallucinations and arithmetic errors. In the petroleum sector, ordering 5,000 liters too much can cause catastrophic underground tank overfill, hazardous spillage, and severe regulatory penalties. Therefore, we adopted a **Deterministic Guardrail Architecture**: specialized mathematical algorithms (LogisticsDecisionEngine) calculate the exact feasible volume based on tank ullage and safety stock equations, while the AI Assistant is restricted purely to explaining the verified results in natural language.

Q3. How is the Stockout Probability mathematically calculated?

Answer: We model lead-time demand uncertainty using a normal Gaussian distribution. The Z-score is calculated as: Z = (Current Stock - Expected Demand during Lead Time) / σ_demand. We then evaluate the cumulative standard normal distribution using the complementary error function: normal_cdf(z) = 0.5 * (1.0 + erf(z / sqrt(2.0))). The stockout probability is 1.0 - normal_cdf(z), representing the exact probability that customer demand exceeds available fuel before delivery.

Q4. What is 'Ullage' and how does the system enforce it?

Answer: Ullage is the remaining volumetric capacity of an underground tank: Ullage = Tank Capacity - Current Inventory. In order_recommendation_agent.py and logistics_engine.py, the system clamps every order quantity to min(recommended_quantity, ullage). Even if demand forecasts recommend a massive order, the system will never permit an order that exceeds ullage.

Q5. How does the frontend handle authentication and session security?

Answer: We implement stateless JWT authentication. When the dealer logs in via /auth/login, the backend issues an HMAC-SHA256 signed JSON Web Token containing the user's ID and dealer ID. The frontend stores this token and uses an Axios request interceptor to attach Authorization: Bearer <token> to every subsequent API request. React Router wraps private views in a ProtectedRoute component that redirects unauthenticated users.

Q6. Explain the difference between Factor Analysis and Cluster Analysis in your project report.

Answer: Cluster Analysis (K-Means) groups *observations (orders)* into distinct behavioral clusters (e.g. Cluster 0: High-Volume Urban, Cluster 1: High-Delay Remote), identifying operational archetypes. In contrast, **Factor Analysis** reduces *variables (features)* by identifying unobserved latent factors (e.g. Traffic Congestion Factor, Inventory Stress Factor) that explain variance across all 350 orders.

Q7. What happens when a dispatch is marked 'DELIVERED' in the database?

Answer: In backend/app/api/dispatch.py, marking a dispatch as delivered triggers a database transaction that: (1) updates dispatch status to DELIVERED with timestamp; (2) resets the assigned tanker status back to AVAILABLE; (3) increments the underground tank inventory: inv.current_level_l = min(inv.tank_capacity_l, inv.current_level_l + order.quantity_l); and (4) commits all changes atomically.

Q8. How does the system prevent dispatching an unqualified driver or unsafe tanker?

Answer: The ComplianceAgent audits each candidate before dispatch approval. It verifies that the driver's license expiry date is greater than the planned arrival date and that their hazardous goods endorsement is active. It also checks that the tanker has not exceeded maximum service mileage and has valid PESO/hydrostatic safety certificates.

Q9. Why did the live stock not change initially when updated in SQLite?

Answer: The platform supports two run environments: local virtual environment (using SQLite) and Docker Compose (using PostgreSQL). The live web application was running inside Docker on port 5173/8000, connected to the PostgreSQL database container (project-db-1). Once updated directly in the PostgreSQL container, the live dashboard immediately reflected the new stock.

Q10. How does the 'What happens if I wait 2 days?' consequence analysis work?

Answer: The LogisticsDecisionEngine projects inventory forward day-by-day: Stock(t+2) = Current Stock - (Forecast Demand × 2). It calculates the resulting stockout probability, tests whether the reserve breaches the safety stock threshold, and provides the dealer with an exact quantitative risk assessment of postponing their replenishment order.

7. Codebase Roadmap: Key Files & File-by-File Walkthrough

If the examiner asks you to open specific files and explain the code, refer to this exact file directory guide:

File Path	Key Functions & Viva Talking Points
backend/app/agents/logistics_engine.py	The central decision brain. Contains LogisticsDecisionEngine, multi-fuel priority scoring, normal_cdf() stockout calculation, ullage clamping, and consequence evaluation ('what if wait 2 days').
backend/app/agents/inventory_agent.py	Computes days of stock, safety stock deficit, and operational risk statuses (NORMAL, WARNING, CRITICAL) for each fuel tank.
backend/app/agents/fleet_intelligence_agent.py	Filters available fleet, matches tanker volume to order demand, and computes fleet capacity utilization percentage.
backend/app/agents/compliance_agent.py	Audits statutory documents against current date, checking for expired licenses, permits, and hazardous materials compliance.
backend/app/models/__init__.py	SQLAlchemy database schema. Contains ORM models: Dealer, User, FuelInventory, FuelOrder, Tanker, Driver, Dispatch.
backend/app/main.py	FastAPI application entry point. Configures CORS middleware, registers API routers, and binds startup lifecycle events.
frontend/src/pages/Dashboard.jsx	React dashboard displaying real-time stock levels, fleet metrics, active dispatches, and executive summary cards.
frontend/src/components/LogisticsDecisionCard.jsx	Visual decision intelligence component showing multi-fuel priority rankings, safety stock buffers, and risk consequence cards.
docker-compose.yml	Production container orchestration definition running PostgreSQL 16 (port 5432), FastAPI (port 8000), and Vite React (port 5173).

Examiner Tip: *When presenting, emphasize the transition from raw transactional data to actionable, explainable decision intelligence with deterministic safety guarantees.*